



**UNIDADE CURRICULAR:** Compilação

**CÓDIGO:** 21018

**DOCENTE:** Constantino e Rudi

**Grupo:** QUADCORE

**João Rodrigues (2203474)**

**Maria Costa (2304361)**

**Nuno Rolo (1900405)**

**Fábio Oliveira (1800960)**

**CURSO:** Licenciatura em Engenharia Informática

**DATA DE ENTREGA:** 05/2026

## RELATÓRIO

## 1. INTRODUÇÃO

O presente relatório descreve o desenvolvimento do compilador para a linguagem MOCP, realizado no âmbito do E-Fólio B da unidade curricular 21018 - Compilação.

O projeto teve como principal objetivo implementar um compilador modular e progressivamente estruturado, capaz de realizar as diferentes fases clássicas de compilação, nomeadamente:

- análise léxica;
- análise sintática;
- construção da árvore sintática abstrata (AST);
- análise semântica;
- geração de código intermédio TAC (Three Address Code);
- otimização do código intermédio.

A solução foi desenvolvida em Python, recorrendo ao ANTLR4 para geração automática do lexer e parser, sendo posteriormente construída uma arquitetura modular composta por múltiplos componentes especializados.

Ao longo do desenvolvimento procurou-se garantir:

- robustez da análise;
- modularidade do código;
- clareza estrutural;
- facilidade de manutenção;
- conformidade com os requisitos da linguagem MOCP;
- deteção controlada de erros;
- separação clara das diferentes fases do compilador.

Foi igualmente dada especial atenção à componente de otimização do código intermédio, incluindo técnicas clássicas como constant folding, propagação de constantes, simplificação algébrica e eliminação conservadora de código morto.

## 2. ARQUITETURA GERAL DO COMPILADOR

O compilador desenvolvido segue uma arquitetura pipeline tradicional, composta pelas seguintes etapas:

1. Leitura do ficheiro fonte;
2. Análise léxica;
3. Análise sintática;
4. Construção da AST;
5. Análise semântica;
6. Geração de TAC;
7. Otimização de TAC.

A execução principal é coordenada através do ficheiro `main.py`, responsável pela integração de todos os módulos do sistema.

A estrutura modular do projeto encontra-se organizada da seguinte forma:

```
main.py
MOCP.g4
ast_nodes.py
ast_builder.py
semantic.py
tac.py
tac_generator.py
optimizer.py
```

error\_handler.py  
run\_tests.py

Cada módulo possui responsabilidades específicas e independentes, promovendo uma arquitetura limpa e facilmente extensível.

### 3. ANÁLISE LÉXICA E SINTÁTICA

A análise léxica e sintática foi implementada com recurso ao ANTLR4.

A gramática da linguagem MOCP foi definida no ficheiro `MOCP.g4`, contendo:

- definição de tokens;
- regras léxicas;
- regras sintáticas;
- precedência de operadores;
- estruturas de controlo;
- funções;
- vetores;
- expressões;
- condições lógicas.

O lexer foi configurado para:

- detetar caracteres inválidos;
- identificar strings não terminadas;
- reconhecer tokens reservados;
- impedir construções proibidas da linguagem C.

A análise sintática foi implementada através do parser gerado automaticamente pelo ANTLR4.

Foi ainda desenvolvido um sistema de recolha de erros através da classe `CollectingErrorListener`, permitindo apresentar mensagens de erro estruturadas e controladas.

Exemplo de erro léxico detetado:

[LÉXICO] linha 18, coluna 8: carácter inesperado '@'.

Exemplo de erro sintático:

[SINTÁTICO] linha 10, coluna 4: missing ';'.

### 4. CONSTRUÇÃO DA AST

Após a análise sintática, a árvore sintática abstrata (AST) é construída através do módulo `ast_builder.py`.

Foi implementado o padrão Visitor, permitindo converter a parse tree produzida pelo ANTLR numa representação abstrata mais limpa e semanticamente relevante.

Foram implementados múltiplos tipos de nós, incluindo:

- `ProgramNode`;
- `FunctionDefNode`;
- `PrototypeNode`;
- `BlockNode`;
- `IfStmtNode`;

- WhileStmtNode;
- ForStmtNode;
- AssignStmtNode;
- FunctionCallNode;
- BinaryExprNode;
- RelationalCondNode;
- BinaryLogicalCondNode;
- ArrayAccessNode;
- CastExprNode.

A AST produzida é posteriormente utilizada:

- pela análise semântica;
- pela geração de código intermédio TAC.

O compilador inclui ainda uma funcionalidade de impressão formatada da AST através do método `pretty()`, facilitando significativamente o debugging e validação estrutural.

## 5. ANÁLISE SEMÂNTICA

A análise semântica foi implementada no módulo `semantic.py`.

Esta fase é responsável pela validação lógica e contextual do programa, garantindo que o código fonte respeita as regras semânticas da linguagem.

Foram implementadas diversas verificações, incluindo:

- utilização de variáveis não declaradas;
- redeclaração de identificadores;
- validação de tipos;
- compatibilidade de atribuições;
- verificação de chamadas de funções;
- validação do número de argumentos;
- verificação de vetores;
- controlo de escopos.

Foi igualmente desenvolvida uma tabela de símbolos hierárquica, permitindo suporte para múltiplos níveis de escopo.

Exemplo de erro semântico detetado:

[SEMÂNTICO] Variável 'y' usada sem declaração prévia.

## 6. GERAÇÃO DE CÓDIGO INTERMÉDIO TAC

Após validação semântica, o compilador gera código intermédio no formato TAC (Three Address Code).

A geração TAC foi implementada no módulo `tac_generator.py`.

O código TAC produzido segue uma representação linear baseada em instruções simples, facilitando:

- análise posterior;
- otimização;
- eventual geração futura de código alvo.

Foram implementadas instruções TAC para:

- atribuições;
- operações aritméticas;
- operadores relacionais;
- chamadas de funções;
- passagem de parâmetros;
- labels;

- saltos condicionais;
- ciclos;
- acessos a vetores;
- casts;
- retornos.

Exemplo de TAC gerado:

```
0113: t21 = 3 * 4
0114: t22 = 2 + t21
0115: ot1 = t22
0116: t23 = ot1 + 0
0117: ot2 = t23
0118: t24 = ot2 * 1
0119: ot3 = t24
0120: t25 = ot3 * 0
0121: ot4 = t25
```

A representação TAC implementada foi desenhada para ser simples, legível e facilmente extensível.

### 6.1 Inicialização a zero

A regra de inicialização por defeito a zero, definida na especificação da MOCP “Sem inicialização (valor por omissão = 0)”, foi aplicada tanto a escalares como a vetores. Nos escalares, esta garantia fica explícita no TAC. Nos vetores, optou-se por não a materializar instrução a instrução, uma vez que isso obrigaria a emitir  $n$  atribuições a zero (uma por elemento) poluindo o TAC. Em alternativa, inclui-se no TAC um comentário que documenta essa premissa, ficando a inicialização efetiva a cargo da fase de geração de código final.

## 7. OTIMIZAÇÃO DE CÓDIGO INTERMÉDIO

Uma das principais extensões implementadas neste E-Fólio foi a criação de um módulo de otimização de TAC.

O módulo `optimizer.py` implementa múltiplas técnicas clássicas de otimização conservadora.

### 7.1 Constant Folding

Foram implementadas otimizações capazes de avaliar expressões constantes durante compilação.

Exemplo:

```
t21 = 3 * 4
```

é automaticamente transformado em:

```
14
```

Esta abordagem reduz operações desnecessárias em tempo de execução.

## 7.2 Simplificação Algébrica

Foram implementadas regras clássicas de simplificação, incluindo:

```
x + 0 → x
x - 0 → x
x * 1 → x
x * 0 → 0
x / 1 → x
x % 1 → 0
```

Estas simplificações permitem reduzir significativamente o número de instruções intermédias.

## 7.3 Constant Propagation

Foi implementada propagação de constantes entre instruções sucessivas.

Exemplo:

```
ot1 = 14
ot2 = ot1
```

é propagado para:

```
ot2 = 14
```

## 7.4 Copy Propagation

Foi igualmente implementada propagação de cópias entre variáveis temporárias.

Esta técnica reduz o número de temporários desnecessários gerados durante a compilação.

## 7.5 Dead Code Elimination

Foi implementada eliminação conservadora de código morto.

O algoritmo remove temporários intermédios que:

- não são utilizados posteriormente;
- não possuem efeitos laterais.

A implementação foi desenhada de forma conservadora, preservando:

- chamadas de funções;
- escrita/leitura;
- labels;
- saltos;
- acessos a vetores;
- retornos.

## 8. DEMONSTRAÇÃO DA OTIMIZAÇÃO

Um dos principais exemplos de otimização obtidos foi o seguinte.

### TAC Original

```
0113: t21 = 3 * 4
0114: t22 = 2 + t21
0115: ot1 = t22
```

```
0116: t23 = ot1 + 0
0117: ot2 = t23
0118: t24 = ot2 * 1
0119: ot3 = t24
0120: t25 = ot3 * 0
0121: ot4 = t25
```

### TAC Otimizado

```
0113: ot1 = 14
0114: ot2 = 14
0115: ot3 = 14
0116: ot4 = 0
```

Pode observar-se claramente:

- avaliação de expressões constantes;
- simplificação algébrica;
- propagação de constantes;
- eliminação de temporários intermédios.

A redução do número de instruções produz um código intermédio significativamente mais eficiente e legível.

## 9. SISTEMA DE TESTES

Foi desenvolvido um conjunto de testes destinado à validação das diferentes componentes do compilador.

Os testes incluem:

- programas válidos;
- programas com erros léxicos;
- programas com erros semânticos;
- programas com ciclos;
- vetores;
- recursividade;
- casts;
- chamadas de funções;
- expressões aritméticas;
- otimizações.

O ficheiro principal de demonstração é:  
`teste_global.mocp`

Foram igualmente criados testes específicos de erro:  
`teste_erros_lexicos.mocp`  
`teste_erros_semanticos.mocp`

Adicionalmente, foi implementado o script:  
`run_tests.py`

que executa automaticamente todos os testes presentes na pasta de testes.

## 10. DIFICULDADES ENCONTRADAS

Durante o desenvolvimento do projeto surgiram diversas dificuldades técnicas.

Uma das principais dificuldades consistiu na integração entre as diferentes fases do compilador, especialmente:

- coerência entre AST e TAC;
- gestão de escopos;
- tratamento de vetores;
- propagação segura de otimizações;
- preservação de efeitos laterais.

A implementação da otimização revelou-se particularmente desafiante, sobretudo na eliminação conservadora de código morto e na propagação de constantes sem comprometer a correção semântica do programa.

Outro aspeto relevante foi a necessidade de garantir modularidade suficiente para permitir evolução futura do compilador.

## 11. CONCLUSÃO

O desenvolvimento do presente compilador permitiu consolidar conhecimentos fundamentais da área de compilação, incluindo:

- análise léxica;
- análise sintática;
- construção de AST;
- análise semântica;
- geração de código intermédio;
- otimização de código.

A solução final apresenta uma arquitetura modular, clara e extensível, sendo capaz de compilar programas MOCP relativamente complexos.

O sistema implementado suporta:

- funções;
- recursividade;
- vetores;
- estruturas de controlo;
- operadores aritméticos;
- operadores relacionais;
- operadores lógicos;
- casts;
- otimização de TAC.

Considera-se que os objetivos propostos para o E-Fólio B foram plenamente cumpridos.

## 12. BIBLIOGRAFIA

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.
- Parr, T. (2013). *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
- Material pedagógico da UC 21018 - Compilação, Universidade Aberta.